ReactJS-HOL 1

Creating the First React Application

Aim

To create and run a simple React application that displays a welcome message.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Install Node.js and NPM.
2. Install Create React App.
3. Create a React application named **myfirstreact**.
4. Open the project in Visual Studio Code.
5. Edit the `App.js` file.
6. Replace the existing code with a heading displaying **"Welcome to the first session of React"**.
7. Run the application using `npm start`.
8. Open the browser and navigate to `http://localhost:3000`.

Sample Output

Welcome to the first session of React

Result

The React application **myfirstreact** was created successfully, and the welcome message was displayed on the browser.

ReactJS-HOL 2

Student Management Portal using Class Components

Aim

To create a React application using class components and render multiple components.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Create a React application named **StudentApp**.
2. Create a folder named **Components** inside the `src` directory.
3. Create three class components:
    - Home
    - About
    - Contact
4. Display the following messages:
    - Home: *Welcome to the Home page of Student Management Portal*
    - About: *Welcome to the About page of the Student Management Portal*
    - Contact: *Welcome to the Contact page of the Student Management Portal*
5. Import all three components into `App.js`.
6. Render the components.
7. Execute the application using `npm start`.

Sample Output

Welcome to the Home Page of Student Management Portal

Welcome to the About Page of the Student Management Portal

Welcome to the Contact Page of the Student Management Portal

Result

The Student Management Portal was successfully created using React class components, and multiple components were rendered on the home page.

ReactJS-HOL 3

Student Score Calculator using Functional Components

Aim

To create a React application using functional components that calculates and displays a student's average score.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Create a React application named **scorecalculatorapp**.
2. Create a folder named **Components**.
3. Create a functional component named **CalculateScore**.
4. Accept the following properties:
    - Name
    - School
    - Total Marks
    - Goal
5. Calculate the student's score percentage.
6. Create a **Stylesheets** folder and add a CSS file (`mystyle.css`) to style the component.
7. Import the component into `App.js`.
8. Pass the required properties to the component.
9. Run the application using `npm start`.

Sample Output

Student Details

Name : Steeve

School : DNV Public School

Total : 284 Marks

Score : 94.67%

Result

The Score Calculator application was successfully created using a React functional component. The student's details and calculated score percentage were displayed with appropriate styling.

ReactJS-HOL 4

Office Space Rental Application using JSX

Aim

To create a React application using JSX to display office space details with conditional styling.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Create a React application named **officespacerentalapp**.
2. Create a JSX element to display the heading of the page.
3. Display an image of the office space using the `img` tag.
4. Create an object containing:
    - Office Name
    - Rent
    - Address
5. Create a list of office objects.
6. Use the `map()` function to display all office details.
7. Apply inline CSS:
    - Display rent in **Red** if it is below **60000**.
    - Display rent in **Green** if it is above **60000**.
8. Run the application using `npm start`.

Sample Output

Office Space Rental

Office Name : Smart Workspaces

Rent : 55000 (Red)

Address : Chennai

Office Name : Tech Park

Rent : 75000 (Green)

Address : Bangalore

Result

The Office Space Rental application was successfully developed using JSX. Office details were displayed dynamically, and conditional styling was applied based on the rent value.

ReactJS-HOL 5

Cricket Application using ES6 Features

Aim

To implement various ES6 features such as `map()`, arrow functions, destructuring, and array merging in a React application.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Create a React application named **cricketapp**.
2. Create a component named **ListOfPlayers**.
3. Store details of 11 cricket players using an array.
4. Display all player details using the `map()` function.
5. Filter players with scores below **70** using arrow functions.
6. Create another component named **IndianPlayers**.
7. Display Odd Team and Even Team players using array destructuring.
8. Create two arrays:
    - T20 Players
    - Ranji Trophy Players
9. Merge both arrays using the spread operator.
10. Display the merged player list.

11. Use a flag variable and an `if-else` statement to render different components.
12. Execute the application using `npm start`.

Sample Output

Player List

Virat Kohli - 92

Rohit Sharma - 88

Shubman Gill - 65

Filtered Players (<70)

Shubman Gill

Odd Team Players

Virat Kohli

KL Rahul

Hardik Pandya

Even Team Players

Rohit Sharma

Jadeja

Bumrah

Merged Players

Virat Kohli

Rohit Sharma

Ashwin

Pujara

Rahane

Result

The Cricket application was successfully created using ES6 features such as `map()`, arrow functions, destructuring, array merging, and conditional rendering.

ReactJS-HOL 9

Event Handling in React

Aim

To implement various event-handling techniques in React using buttons, synthetic events, and a currency converter.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Create a React application named **eventexamplesapp**.
2. Create **Increment** and **Decrement** buttons to modify the counter value.
3. Invoke multiple methods when the Increment button is clicked.
4. Display a greeting message using the **Say Hello** function.
5. Create a **Say Welcome** button that accepts an argument.
6. Implement a synthetic event that displays **"I was clicked"**.
7. Create a **CurrencyConvertor** component.
8. Convert Indian Rupees to Euro when the Convert button is clicked.
9. Handle the button click using the `handleSubmit()` event.
10. Run the application using `npm start`.

Sample Output

Counter : 10

Increment

Counter : 11

Hello! Have a nice day.

Welcome

Welcome User!

I was clicked

Currency Converter

Amount : ₹1000

Euro : €10.80

Result

The React application successfully demonstrated event handling, synthetic events, multiple event methods, and currency conversion using React event handlers.

ReactJS-HOL 10

Styling React Components using CSS Modules

Aim

To style React components using CSS Modules and inline styles.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Download and open the provided React application.
2. Restore the required node packages.
3. Open the project in Visual Studio Code.
4. Create a CSS Module named **CohortDetails.module.css**.
5. Define a CSS class named **box** with the required styling:
    - Width: 300px
    - Display: inline-block
    - Margin: 10px
    - Padding: 10px (top & bottom), 20px (left & right)
    - 1px solid black border
    - Border radius: 10px

6. Apply the **box** class to the container component.
7. Style the `<dt>` element using a tag selector.
8. Display the cohort status in:
    - Green for **Ongoing**
    - Blue for **Completed**
9. Execute the application using `npm start`.

Sample Output

Cohort Details

React Training

Status : Ongoing (Green)

Trainer : John

Duration : 8 Weeks

Java Training

Status : Completed (Blue)

Trainer : David

Duration : 6 Weeks

Result

The React application was successfully styled using CSS Modules and inline styling. Different colors were displayed based on the cohort status.

ReactJS-HOL 11

React Component Lifecycle Methods

Aim

To implement React lifecycle methods such as `componentDidMount()` and `componentDidCatch()`.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js
- Fetch API

Procedure

1. Create a React application named **blogapp**.
2. Create a `Post` class.
3. Create a class component named **Posts**.
4. Initialize the component state with an empty list.
5. Create a `loadPosts()` method.

Fetch post data from:
https://jsonplaceholder.typicode.com/posts

6.
7. Call `loadPosts()` inside the `componentDidMount()` lifecycle method.
8. Display the title and body of each post.
9. Implement `componentDidCatch()` to handle runtime errors.
10. Add the `Posts` component to `App.js`.
11. Execute the application.

Sample Output

Posts

Post Title 1

This is the first post.

Post Title 2

This is the second post.

Post Title 3

This is the third post.

Result

The React application successfully demonstrated the use of lifecycle methods by fetching data using `componentDidMount()` and handling errors using `componentDidCatch()`.

ReactJS-HOL 12

Ticket Booking Application using Conditional Rendering

Aim

To implement conditional rendering in React by displaying different pages for guest and logged-in users.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Create a React application named **ticketbookingapp**.
2. Create components for:
    - Guest User
    - Logged-in User
3. Display flight details to guest users.
4. Allow only logged-in users to book tickets.
5. Create **Login** and **Logout** buttons.
6. Use conditional rendering to switch between Guest and User pages.
7. Run the application using `npm start`.

Sample Output

Guest Page

Available Flights

Chennai → Delhi

Chennai → Mumbai

[Login]

User Page

Welcome User

Book Your Flight

[Logout]

Result

The Ticket Booking application was successfully developed using conditional rendering. Different pages were displayed based on the user's login status.

ReactJS-HOL 13

Conditional Rendering with Lists and Keys

Aim

To implement conditional rendering using multiple components, lists, keys, and the `map()` function in React.

Software Requirements

- Node.js
- NPM
- Visual Studio Code
- React.js

Procedure

1. Create a React application named **bloggerapp**.
2. Create three components:
    - Book Details
    - Blog Details
    - Course Details
3. Display each component using different conditional rendering techniques.
4. Render lists using the `map()` function.
5. Assign unique keys while rendering list items.
6. Use techniques such as:
    - `if-else`

    - Ternary Operator
    - Logical `&&`
    - Element Variables
7. Execute the application using `npm start`.

Sample Output

Book Details

Book Name : React Complete Guide

Author : Max Schwarzmüller

Price : ₹599

Blog Details

Title : Introduction to React

Author : John Doe

Published : Yes

Course Details

Course : ReactJS

Duration : 8 Weeks

Trainer : Cognizant

Result

The Blogger application was successfully developed using conditional rendering techniques. Multiple components were rendered using lists, keys, and the `map()` function.